Restricting cotrees faithfully to induced subgraphs

30 October 2026
Source-repair working note

This note proves the faithful representation step for hereditary classes of cographs and records the small obstructions that a decision argument must respect. The accompanying decision proof uses a finite cover by ranked-tree ideals and effective inclusion of their graph images. Ordinary topological tree containment is not substituted for induced graph containment.

1 The representation and its restriction

A cotree is either empty, a leaf carrying a vertex name, or a node with label 0 or 1 and at least two children. Leaf names in a tree are distinct. The children are unordered. Its graph $\mathrm{G}(T)$ has the leaves as vertices. A node with label 0 takes the disjoint union of its child graphs; label 1 takes their join, adding every edge between different child graphs. The empty cotree represents the empty graph. A nonempty cotree is *reduced* if no internal node has the same label as its parent. These are operation labels; leaf names are not changed during restriction.

For a set A of retained leaf names, define $R_A(T)$ recursively. Delete each leaf outside A. At an internal node first restrict all its children and discard empty results. If no child remains, return the empty cotree; if one remains, return that child, with its own label. Otherwise flatten each child whose operation label equals the current node's label: replace that child by its children. Retain the current operation and the resulting list. In particular, an edge between *different* operation labels is never contracted while retaining both branching operations' leaves.

Lemma 1. *For every cotree T and every set A of its leaves, $R_A(T)$ is reduced, has precisely the leaves A, and satisfies the literal equality*

$$\mathrm{G}(R_A(T)) = \mathrm{G}(T)[A].$$

If T is reduced and A is its whole leaf set, $R_A(T) = T$ up to the irrelevant order of children.

Proof. Induct on T. The empty and single-leaf cases follow from the definition. Suppose its root operation is $a \in \{0, 1\}$, with children T_i, and put $A_i = A \cap V(\mathrm{G}(T_i))$. By induction each retained child represents exactly $\mathrm{G}(T_i)[A_i]$. Restricting either a disjoint union or a join to A is the same operation on these child restrictions: within a child the edges are unchanged, and between different children all edges are absent if $a = 0$ and present if $a = 1$.

The empty graph is an identity for both operations, and an operation on one graph returns that graph. Thus the zero-child and one-child cases preserve this equality. Both operations are associative, so flattening a child with the *same* label also preserves every edge. This proves the graph identity and the exact leaf set.

The restricted children were reduced. A same-labeled child's children have the opposite operation label, or are leaves. After flattening, therefore, no child has the root's label, and the arity remains at least two. The zero-child and one-child cases are already reduced by induction. Finally, on a reduced tree retaining all leaves no child disappears or is flattened, so the last assertion follows by the same induction. □

Lemma 2. *Two reduced cotrees represent isomorphic graphs if and only if they are isomorphic as operation-labeled trees, with the induced bijection of leaves.*

Proof. The reverse implication follows directly from evaluation. For the forward implication induct on the number of vertices; zero and one are immediate. A reduced root labeled 0 has connected child graphs: each child is a leaf or has a join root, and a join of at least two nonempty graphs is connected. Thus its children are exactly the connected components of its graph. A root labeled 1 gives a connected graph; its children are exactly the connected components of the complement. Indeed, complementation exchanges the two operations, and reduces this assertion to the previous case.

Consequently the graph determines the root operation (disconnected or connected), and determines the unordered vertex partition into children, using components or complement-components respectively. A graph isomorphism preserves that partition and the induced child graphs. Apply induction in each child to obtain the tree isomorphism. This proof also

gives a finite algorithm for reconstructing the reduced cotree of a graph already known to have such a representation. □

Theorem 3 (Faithful containment)**.** *For reduced cotrees S, T, an induced graph embedding $f : \mathrm{G}(S) \hookrightarrow \mathrm{G}(T)$ exists if and only if some leaf set $A \subseteq V(\mathrm{G}(T))$ has $R_A(T)$ isomorphic to S. More precisely, the given leaf injection f is an induced graph embedding exactly when its leaf bijection extends to an isomorphism $S \cong R_{f(V(\mathrm{G}(S)))}(T)$.*

Proof. An induced graph embedding is an isomorphism onto the induced subgraph on its image. Lemma 1 identifies that graph with the evaluation of the restricted reduced cotree, and Lemma 2 identifies its reduced cotree, with the prescribed leaf bijection. The converse follows by evaluating a cotree isomorphism and then including the retained vertices in $\mathrm{G}(T)$. □

Equivalently, an injective leaf map must preserve the operation label of the lowest common ancestor of *each pair of distinct leaves*. It need not preserve the lowest common ancestor vertex itself. The theorem gives a direct terminating containment test by finite leaf selection and normalization. It does not yet give a uniform bound on a first bad pair.

2 The two small source obstructions

Write U for a union node, J for a join node, and use letters for leaves. The cotree

$$U(a, J(b, U(c, d)))$$

represents an isolated vertex a and a three-vertex path with centre b. Retaining a, c, d deletes b, suppresses its now-unary join, and flattens the two remaining union nodes. The result is $U(a, c, d)$, three isolated vertices. A downward edge-disjoint-path embedding of this three-child star into the displayed binary target does not exist: the star's three outgoing edges would require three different outgoing target branches at its root image. This is precisely where strict topological containment is too strong.

Conversely,

$$U(a, J(b, c))$$

represents an isolated vertex and an edge. Contracting the two differently labeled internal nodes and retaining the parent label would yield $U(a, b, c)$, which represents three isolated vertices. No induced embedding of the latter graph into the former exists: the graphs have the same three vertices in

number, but one has an edge and the other does not. Our restriction retains the join node when all three leaves are retained and does not allow this contraction.

These examples address the cotree step in Carter's *On the joint embedding property for cographs and trees*, arXiv:2409.06127v1, Section 3.2, under the two stated readings. They do not constitute counterexamples to his final decidability conclusion.

3 Why binary refinements alone do not repair joint embedding

It is tempting to replace each reduced cotree by all of its binary parses and apply a topological-tree JEP algorithm. The following obstruction shows that a further proof would be essential.

Proposition 4. *The graph class consisting of the independent graphs with at most four vertices has JEP. Its language of all full binary union-only parses does not have JEP under strict label-preserving topological containment.*

Proof. Two independent graphs of sizes $r, s \leq 4$ embed into the independent graph of size $\max(r, s)$. On the tree side consider the balanced and comb parses

$$U(U(a,b),U(c,d)), \qquad U(a,U(b,U(c,d))).$$

Both represent the same four-vertex independent graph, but they are non-isomorphic full binary trees. Any host in the language has at most four leaves. An embedding of either source uses four distinct leaves, so such a host has exactly four leaves and seven vertices. The source also has seven vertices; its injective vertex map is therefore bijective. The six source edges require six nonempty edge-disjoint target paths in a target with six edges. Each path is a single edge. Hence either embedding must be an isomorphism, which cannot hold for both shapes in one host. Smaller hosts cannot contain either source. □

4 The remaining decision obligation

The useful binary relation

The failure of joint embedding to transfer through arbitrary binary representations does not make binary trees useless. Use the ranked alphabet

$\{a : 0, U : 2, J : 2\}$, with its *ordered homeomorphic* relation $\preceq$: a source may embed into one child of a target, or equal root symbols may be coupled with childwise embeddings. Here leaf vertex names are irrelevant; each occurrence of a is a separate graph vertex.

Lemma 5. *Evaluation is monotone from this binary relation to induced graph containment. Moreover, for a binary parse T and any nonempty set A of its leaves, there is a binary parse $P_A(T) \preceq T$ whose evaluation is precisely $\mathrm{G}(T)[A]$.*

Proof. For monotonicity, the dive case includes the graph of a child as an induced subgraph. In the coupling case use the two child embeddings on disjoint vertex sets; the common operation adds exactly the required cross-edges, all or none. Induction on the homeomorphic embedding proves the assertion.

For pruning, delete leaves outside A, discard empty subtrees, and suppress unary nodes, but do not flatten same-labeled binary nodes. The graph identity follows by the union/join induction in Lemma 1. The result is binary. When both children survive, the coupling rule and the inductive embeddings apply. When only one survives, the dive rule applies. Thus the resulting nonempty parse homeomorphically embeds in T. □

An ideal is a nonempty downward-closed directed set. If I is an ideal of binary parses, its graph image, with the empty graph adjoined, is a graph ideal: monotonicity proves directedness and Lemma 5 proves heredity. This proves substantially more than an arbitrary computable map of representations would provide.

Lemma 6 (Finite ideal covers)**.** *If a nonempty hereditary graph class $\mathcal{K}$ is covered by finitely many graph ideals $D_1, \ldots, D_r \subseteq \mathcal{K}$, then it has JEP if and only if $\mathcal{K} = D_i$ for some i.*

Proof. Equality with an ideal implies JEP. Conversely, if every D_i is a proper subset, choose $G_i \in \mathcal{K} \setminus D_i$. Iterated JEP gives one $H \in \mathcal{K}$ containing all the G_i. Heredity of each D_i implies $H \notin D_i$ for every i, contradicting the cover. The empty-class case is handled separately and has vacuous JEP. □

Thus a decision route can use a finite *cover* by binary ideals without requiring the whole binary language itself to have JEP. Its graph images may even coincide. This distinction accommodates the independent-set example above.

What still needs effectivity

For paper 03 the forest-to-comparability-graph correspondence, width classification, finite negative-certificate search and age construction remain separate from the general positive decision guarantee. Theorem 3 repairs the semantics of induced graph containment. The accompanying decision argument constructs a total procedure using the finite ideal-cover criterion above, effective ranked-tree ideal decomposition, and exact Parikh/Presburger tests for graph-image inclusion. Its full proof is given in Section 3 of the canonical manuscript and in the decision-repair working proof. It does not rely on the disputed graph corollary.

Regular recognition of the permitted individual graphs is not enough: the algorithm must decide the existence of a common permitted host for arbitrarily large input pairs, and then the absence of every bad pair. Neither the finite leaf-selection test nor Proposition 4 supplies that uniform termination guarantee. The effective ideal-decomposition and inclusion steps in the accompanying proof provide it. The executable controls in this directory check the semantic representation and state-set lemmas on bounded inputs; they are not an implementation or formal verification of the full algorithm.